

**Exp 4: IDENTIFY KEY PRESS IN KEYPAD
MATRIX AND PRINT THE SCAN VALUE
TO THE CONSOLE**

**Name: Pradip Maity
CDSID: PMAITY**



Table of Contents

S.No.		Page No
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	3
5	Working Principle	4
6	Application Code	4
7	Output	5
8	Hardware Image	5
9	Appendix	6

1. Introduction:

This project demonstrates how to detect key presses from a matrix keypad using the Raspberry Pi Pico. A matrix keypad consists of rows and columns arranged in a grid format, where each key is identified by a unique row and column combination.

Keypad scanning is a technique used to detect which key is pressed by sequentially activating rows and reading column inputs. The detected key value is then transmitted to a UART serial console for display.

2. Scope:

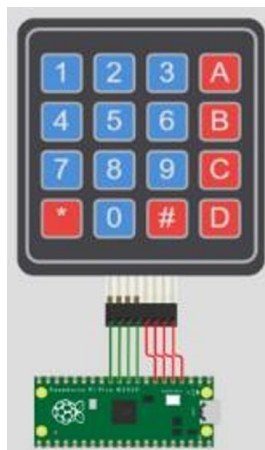
This project provides an understanding of interfacing input devices like matrix keypads with a microcontroller. It introduces concepts such as GPIO control, scanning techniques, and user input handling. These concepts are widely used in embedded systems for applications like password entry systems, control panels, and user interfaces.

3. Prerequisite:

Hardware: Raspberry Pi Pico Microcontroller, USB Connector, Breadboard, Matrix Keypad (4x4), Jumper Wires

Software: VS Code (with Pico SDK), PuTTY Serial Console (in Windows)

4. Connection Diagram:



5. Working Principle:

The matrix keypad is connected to GPIO pins of the Raspberry Pi Pico, with rows configured as output pins and columns configured as input pins with pull-up resistors.

During scanning, one row is set to LOW at a time while all other rows remain HIGH. The columns are then read to check if any key in the active row is pressed. When a key is pressed, it connects the corresponding row and column, causing the column input to go LOW.

By identifying the active row and the detected column, the exact key pressed is determined using a predefined key mapping. A small delay is used for debouncing to avoid false detection.

Once a valid key press is detected, the corresponding value is sent through UART and displayed on the PuTTY serial console. This process repeats continuously to detect every key press in real time.

6. Application code:

```
#include <stdio.h>
#include "pico/stdlib.h"

const uint ROWS[4] = {2, 3, 4, 5};
const uint COLS[4] = {6, 7, 8, 9};
char keys[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

void keypad_init() {
    for (int i = 0; i < 4; i++) {
        gpio_init(ROWS[i]);
        gpio_set_dir(ROWS[i], GPIO_OUT);
        gpio_put(ROWS[i], 1);
    }
    for (int i = 0; i < 4; i++) {
        gpio_init(COLS[i]);
        gpio_set_dir(COLS[i], GPIO_IN);
        gpio_pull_up(COLS[i]);
    }
}
```

```
char scan_keypad() {
    for (int row = 0; row < 4; row++) {
        for (int i = 0; i < 4; i++)
            gpio_put(ROWS[i], 1);
        gpio_put(ROWS[row], 0);
        sleep_ms(5);

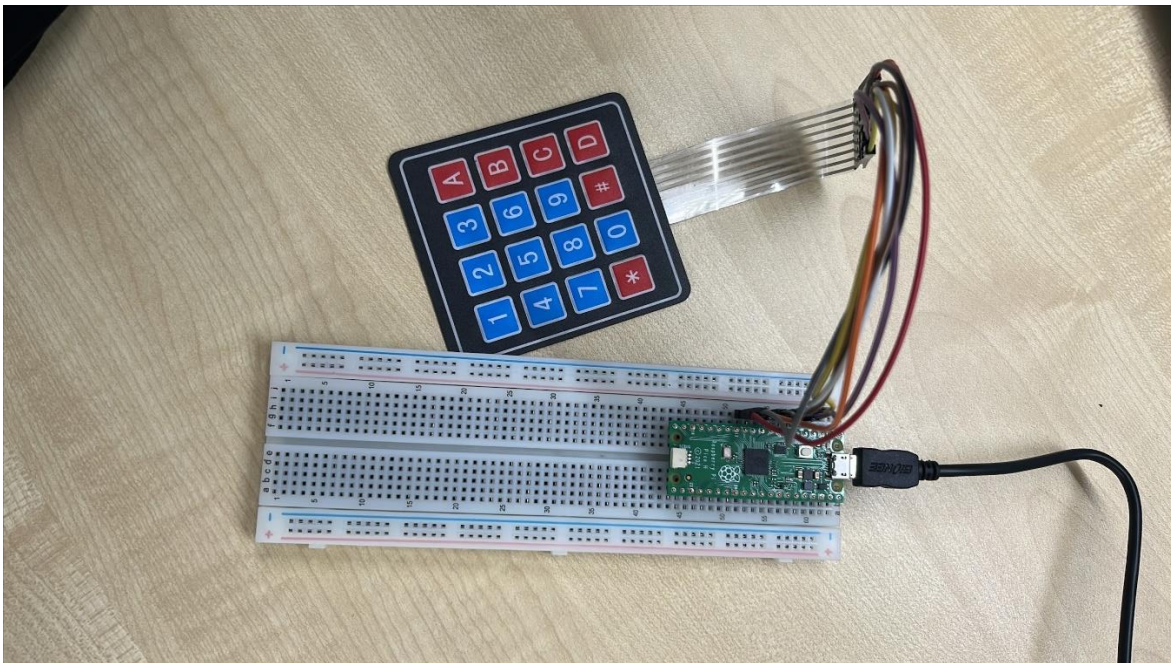
        for (int col = 0; col < 4; col++) {
            if (gpio_get(COLS[col]) == 0) {
                sleep_ms(20);
                while (gpio_get(COLS[col]) == 0);
                return keys[row][col];
            }
        }
    }
    return 0;
}

int main() {
    stdio_init_all();
    keypad_init();
    printf("Keypad Ready...\n");
    while (1) {
        char key = scan_keypad();
        if (key != 0) {
            printf("Key Pressed: %c\n", key);
        }
        sleep_ms(50);
    }
}
```

7. Output:

```
COM4 - PuTTY
Key Pressed: 1
Key Pressed: 2
Key Pressed: 3
Key Pressed: A
Key Pressed: 4
Key Pressed: 5
Key Pressed: 6
Key Pressed: B
Key Pressed: 7
Key Pressed: 8
Key Pressed: 9
Key Pressed: C
Key Pressed: *
Key Pressed: 0
Key Pressed: #
Key Pressed: D
```

8. Hardware Image:



9. Appendix:

References

- Official Documentation for Raspberry Pi Pico and Pico W
- Technical Datasheet for Raspberry Pi Pico
- GitHub Repository for Raspberry Pi Pico SDK
- PuTTY Serial Console Documentation